Quantum Circuit Group Assignments (30 Points)

80 นักศึกษา ÷ กลุ่มละ 2-3 คน = 40 หัวข้อไม่ซ้ำกัน

กติกากลาง (ทุกกลุ่ม)

ใช้โค้ด qiskit_bloch.py เป็น template — แก้ไข layers, NUM_QUBITS ตามหัวข้อของกลุ่ม

แสดง Bloch sphere + Statevector ในแต่ละ depth layer

เขียนรายงานอธิบาย ว่าแต่ละ layer ทำอะไร ทำไม Bloch sphere เปลี่ยน

นำเสนอหน้าชั้นเรียน กลุ่มละ 10 นาที

ห้ามซ้ำกัน — แต่ละกลุ่มได้วงจรคนละแบบ

สิ่งที่ต้องส่ง

โค้ด Python (.py)

ภาพ Bloch sphere ทุก layer

รายงาน 2-3 หน้า (อธิบายวงจร + วิเคราะห์ผลลัพธ์)

เกณฑ์	คะแนน
โค้ดถูกต้อง รันได้ แสดง Bloch sphere ครบ	10
คำอธิบายทางฟิสิกส์ถูกต้อง ชัดเจน	10
วิเคราะห์ Bloch sphere + entanglement	10

ตารางจับฉลากกลุ่ม

กลุ่ม	หัวข้อ	Qubits
1	Hadamard Sandwich	2
2	Rotation Staircase	2
3	Phase Accumulation	2
4	XYZ Axis Tour	1
5	Identity from Gates	2
6	T Gate Decomposition	2
7	Superposition Explorer	2
8	Pauli Gate Algebra	1
9	Root-of-NOT	2
10	Bloch Sphere Equator Walk	1
11	Bell State Φ+	2
12	Bell State Φ−	2
13	Bell State Ψ+	2
14	Bell State Ψ−	2
15	GHZ State	3
16	W State	3
17	Entangle & Disentangle	2
18	CZ vs CX Entanglement	2
19	Partial Entanglement	3
20	Chain Entanglement	3
21	Quantum NOT via CX	2
22	SWAP from 3 CX	2
23	Deutsch Oracle (balanced)	2

กลุ่ม	หัวข้อ	Qubits
24	Deutsch Oracle (constant)	2
25	Phase Kickback	2
26	Quantum Interference	1
27	Amplitude Amplification	2
28	QFT (2 qubits)	2
29	Toffoli Preparation	3
30	Fredkin (CSWAP)	3
31	Quantum RNG	3
32	Dense Coding	2
33	Bit Flip Error Detection	3
34	Phase Flip Error Detection	3
35	Quantum Coin Flip	2
36	Quantum Adder	3
37	Entanglement Swapping	3
38	Quantum Classifier	2
39	Bernstein-Vazirani	3
40	Simon's Problem	4

กลุ่มที่ 1-10: Single-Qubit Gate Sequences (2 qubits, depth 4)

โจทย์: ใช้ 2 qubits, ออกแบบ 4 layers ตามหัวข้อ แสดง Bloch sphere ทุก layer เน้น: ดูการเคลื่อนที่ของ Bloch vector บน sphere

กลุ่ม 1 — Hadamard Sandwich

python

```
NUM_QUBITS = 2
layers = [
    [("h", [0]), ("h", [1])],          # Layer 1: H ทั้งคู่
    [("z", [0]), ("x", [1])],          # Layer 2: Z, X
    [("h", [0]), ("h", [1])],          # Layer 3: H อีกรอบ
    [("x", [0]), ("z", [1])],          # Layer 4: สลับ gate
]
```

คำถาม: ทำไม H⟶Z⟶H ได้ผลเหมือน X gate? พิสูจน์ด้วย Bloch sphere

กลุ่ม 2 — Rotation Staircase

python

```
NUM_QUBITS = 2
layers = [
    [("rx", [0], np.pi/8),  ("ry", [1], np.pi/8)],
    [("rx", [0], np.pi/4),  ("ry", [1], np.pi/4)],
    [("rx", [0], np.pi/2),  ("ry", [1], np.pi/2)],
    [("rx", [0], np.pi),    ("ry", [1], np.pi)],
]
```

คำถาม: RX หมุนรอบแกนไหน? RY หมุนรอบแกนไหน? เทียบ path บน Bloch sphere

กลุ่ม 3 — Phase Accumulation

python

```
NUM_QUBITS = 2
layers = [
    [("h", [0]), ("h", [1])],
    [("s", [0]), ("t", [1])],          # S = π/2, T = π/4
    [("s", [0]), ("t", [1])],          # สะสม phase อีกรอบ
    [("s", [0]), ("t", [1])],          # สะสม phase อีก
]
```

คำถาม: Phase สะสมขึ้นเรื่อยๆ อย่างไร? S×S×S เท่ากับ gate อะไร? T×T เท่ากับอะไร?

กลุ่ม 4 — XYZ Axis Tour

python

```
NUM_QUBITS = 1
layers = [
    [("ry", [0], np.pi/2)],            # |0⟩ ⟶ |+⟩ (X-axis)
    [("rx", [0], np.pi/2)],            # |+⟩ ⟶ ??? (Y-axis region)
    [("rz", [0], np.pi/2)],            # rotate around Z
    [("ry", [0], -np.pi/2)],           # bring back
]
```

คำถาม: วาด path ที่ Bloch vector เดินทางบน sphere ตั้งแต่ Layer 1-4

กลุ่ม 5 — Identity from Gates

python

```
NUM_QUBITS = 2
layers = [
    [("h", [0]), ("x", [1])],
    [("x", [0]), ("x", [1])],
    [("h", [0]), ("x", [1])],        # ⟵ จะกลับมาเหมือนเดิมไหม?
    [("z", [0]), ("x", [1])],
]
```

คำถาม: ชุด gate ไหนที่ทำให้ qubit กลับมาสถานะเดิม $|0\rangle$? ทำไม?

กลุ่ม 6 — T Gate Decomposition

```
python
NUM_QUBITS = 2
# q0: ใช้ T gate 8 ครั้ง, q1: ใช้ S gate 4 ครั้ง
layers = [
    [("t", [0]), ("t", [0]), ("s", [1])],
    [("t", [0]), ("t", [0]), ("s", [1])],
    [("t", [0]), ("t", [0]), ("s", [1])],
    [("t", [0]), ("t", [0]), ("s", [1])],
]
```

คำถาม: T^8 = ? , S^4 = ? พิสูจน์ด้วย Bloch sphere ว่ากลับมาจุดเดิม

กลุ่ม 7 — Superposition Explorer

```
python
NUM_QUBITS = 2
layers = [
    [("ry", [0], np.pi/6), ("ry", [1], np.pi/3)],    # superposition ไม่เท่ากัน
    [("rz", [0], np.pi/3), ("rz", [1], np.pi/6)],    # เพิ่ม phase
    [("ry", [0], np.pi/3), ("ry", [1], np.pi/6)],    # หมุนเพิ่ม
    [("h",  [0]),          ("h",  [1])],             # Hadamard ปิดท้าย
]
```

คำถาม: Probability distribution เปลี่ยนอย่างไรในแต่ละ layer? layer ไหนกระจายมากสุด?

กลุ่ม 8 — Pauli Gate Algebra

```
python
NUM_QUBITS = 1
layers = [
    [("x", [0])],          # X|0⟩ = |1⟩
    [("y", [0])],          # Y|1⟩ = ?
    [("z", [0])],          # Z on result = ?
    [("x", [0])],          # X on result = ?
]
```

คำถาม: คำนวณมือ XZY $X|0\rangle$ ทีละขั้น แล้วเทียบกับ Bloch sphere ว่าตรงกันไหม

กลุ่ม 9 — Root-of-NOT

```
python
NUM_QUBITS = 2
# √X gate = RX(π/2) ทำสองครั้งจะได้ X
layers = [
```

```
    [("rx", [0], np.pi/2), ("rx", [1], np.pi/2)],    # √X ครั้งที่ 1
    [("rx", [0], np.pi/2), ("rz", [1], np.pi/2)],    # √X อีกรอบ vs √Z
    [("rx", [0], np.pi/2), ("rx", [1], np.pi/2)],    # √X ครั้งที่ 3
    [("rx", [0], np.pi/2), ("rz", [1], np.pi/2)],    # √X ครั้งที่ 4 vs √Z
]
```

คำถาม: √X ทำ 2 ครั้ง = X จริงไหม? √Z ทำ 2 ครั้ง = Z จริงไหม? ดูจาก Bloch sphere

กลุ่ม 10 — Bloch Sphere Equator Walk

python

```
NUM_QUBITS = 1
layers = [
    [("h",  [0])],                  # ไปที่ equator: |+⟩
    [("rz", [0], np.pi/2)],          # เดินบน equator 90°: |+i⟩
    [("rz", [0], np.pi/2)],          # เดินอีก 90°: |−⟩
    [("rz", [0], np.pi/2)],          # เดินอีก 90°: |−i⟩
]
```

คำถาม: วาดรูป path บน equator ของ Bloch sphere ทั้ง 4 จุด คือสถานะอะไร?

กลุ่มที่ 11-20: Entanglement Circuits (2-3 qubits, depth 4)

โจทย์: สร้าง entanglement ด้วย CX/CZ แล้ววิเคราะห์ค่า r ที่ลดลง เน้น: เข้าใจว่า r < 1 หมายถึง entanglement

กลุ่ม 11 — Bell State |Φ+⟩

python

```
NUM_QUBITS = 2
layers = [
    [("h",  [0])],
    [("cx", [0, 1])],
    [("rz", [0], np.pi/4)],          # เพิ่ม phase หลัง entangle
    [("h",  [1])],                  # H บน qubit ที่ entangle
]
```

คำถาม: r ของทั้งสอง qubit เปลี่ยนอย่างไรหลัง CX? Layer 4 ทำให้ disentangle ได้ไหม?

กลุ่ม 12 — Bell State |Φ−⟩

python

```
NUM_QUBITS = 2
layers = [
    [("h",  [0])],
    [("z",  [0])],                  # เพิ่ม Z ก่อน CX
    [("cx", [0, 1])],
    [("rz", [0], np.pi/3), ("ry", [1], np.pi/6)],
]
```

คำถาม: |Φ−⟩ ต่างจาก |Φ+⟩ อย่างไร? Bloch sphere บอกความแตกต่างได้ไหม?

กลุ่ม 13 — Bell State |Ψ+⟩

python

```
NUM_QUBITS = 2
layers = [
    [("h",  [0])],
```

```
    [("x",  [1])],                  # flip q1 ก่อน
    [("cx", [0, 1])],
    [("s",  [0]), ("t", [1])],
]
```

คำถาม: ทำไมต้อง X(q1) ก่อน CX? statevector ต่างจาก $|\Phi+\rangle$ อย่างไร?

กลุ่ม 14 — Bell State $|\Psi-\rangle$

python

```
NUM_QUBITS = 2
layers = [
    [("h",  [0])],
    [("x",  [1])],
    [("cx", [0, 1])],
    [("z",  [0])],                  # Z สร้าง minus sign
]
```

คำถาม: เปรียบเทียบ statevector ของ Bell State ทั้ง 4 แบบ (ใช้ผลจากกลุ่ม 11-14 มารวมกัน)

กลุ่ม 15 — GHZ State (3 qubits)

python

```
NUM_QUBITS = 3
layers = [
    [("h",  [0])],
    [("cx", [0, 1])],
    [("cx", [1, 2])],
    [("rz", [0], np.pi/6), ("rz", [1], np.pi/6), ("rz", [2], np.pi/6)],
]
```

คำถาม: r ของทุก qubit เท่ากันไหม? ทำไม? GHZ ต่างจาก Bell State อย่างไร?

กลุ่ม 16 — W State (3 qubits)

python

```
NUM_QUBITS = 3
# |W⟩ = (|001⟩ + |010⟩ + |100⟩) / √3
layers = [
    [("ry", [0], 2*np.arccos(np.sqrt(1/3)))],       # Layer 1
    [("cx", [0, 1]), ("ry", [1], np.pi/4)],         # Layer 2 (approximate)
    [("cx", [1, 2])],                               # Layer 3
    [("x",  [0])],                                  # Layer 4: adjust
]
```

คำถาม: W State ต่างจาก GHZ State อย่างไร? ค่า r ของแต่ละ qubit เท่ากันไหม?

กลุ่ม 17 — Entangle แล้ว Disentangle

python

```
NUM_QUBITS = 2
layers = [
    [("h",  [0])],                  # Layer 1: superposition
    [("cx", [0, 1])],               # Layer 2: entangle ⟶ r=0
    [("cx", [0, 1])],               # Layer 3: CX อีกรอบ (undo?)
    [("h",  [0])],                  # Layer 4: H อีกรอบ (undo?)
```

```
]
```

คำถาม: CX สองครั้งติดกัน cancel กันไหม? r กลับเป็น 1 หรือไม่? พิสูจน์

กลุ่ม 18 — CZ vs CX Entanglement

python

```
NUM_QUBITS = 2
layers = [
    [("h",  [0]), ("h",  [1])],       # ทั้งคู่ superposition
    [("cz", [0, 1])],                 # ใช้ CZ แทน CX
    [("h",  [0])],                    # H หลัง CZ
    [("h",  [1])],                    # H อีกตัว
]
```

คำถาม: CZ สร้าง entanglement ต่างจาก CX อย่างไร? ต้อง H ก่อน/หลังอย่างไร?

กลุ่ม 19 — Partial Entanglement

python

```
NUM_QUBITS = 3
layers = [
    [("h",  [0]), ("h",  [1]), ("h",  [2])],
    [("cx", [0, 1])],                 # entangle แค่ q0-q1
    [("rz", [2], np.pi/3)],           # q2 ยังอิสระ
    [("s",  [0]), ("t",  [1]), ("h",  [2])],
]
```

คำถาม: q2 มี r เท่าไหร่? ทำไมต่างจาก q0, q1? อธิบายด้วย partial trace

กลุ่ม 20 — Chain Entanglement

python

```
NUM_QUBITS = 3
layers = [
    [("h",  [0])],
    [("cx", [0, 1])],                 # entangle q0-q1
    [("cx", [1, 2])],                 # entangle q1-q2
    [("cx", [0, 2])],                 # entangle q0-q2 ด้วย
]
```

คำถาม: หลังทำ CX สามครั้ง r ของแต่ละ qubit เป็นเท่าไหร่? เปรียบเทียบกับ GHZ State

---

กลุ่มที่ 21-30: Quantum Algorithm Building Blocks (2-3 qubits)

โจทย์: implement ส่วนสำคัญของ quantum algorithm จริง เน้น: เข้าใจบทบาทของแต่ละ gate ผ่าน Bloch sphere

กลุ่ม 21 — Quantum NOT (bit flip ด้วย CX)

python

```
NUM_QUBITS = 2
layers = [
    [("ry", [0], np.pi/3)],           # q0 = arbitrary state
    [("x",  [1])],                    # q1 = |1⟩ (ancilla)
    [("cx", [0, 1])],                 # flip q1 based on q0
    [("x",  [1])],                    # correct
]
```

คำถาม: q1 สุดท้ายมี Bloch vector เหมือน q0 ตอนเริ่มต้นไหม? ทำไม?

กลุ่ม 22 — Swap Gate จาก CX สามตัว

python

```
NUM_QUBITS = 2
layers = [
    [("ry", [0], np.pi/3), ("rx", [1], np.pi/4)],   # สถานะต่างกัน
    [("cx", [0, 1])],                # CX ตัวที่ 1
    [("cx", [1, 0])],                # CX ตัวที่ 2
    [("cx", [0, 1])],                # CX ตัวที่ 3
]
```

คำถาม: หลัง 3 CX Bloch vector ของ q0 กับ q1 สลับกันจริงไหม? เทียบกับ Layer 1

กลุ่ม 23 — Deutsch's Oracle (f(x) = x)

python

```
NUM_QUBITS = 2
layers = [
    [("x",  [1])],                # q1 = |1⟩
    [("h",  [0]), ("h",  [1])],        # superposition ทั้งคู่
    [("cx", [0, 1])],                # Oracle: f(x)=x
    [("h",  [0])],                # Hadamard q0 ⟶ วัดผล
]
```

คำถาม: หลัง Layer 4 q0 อยู่ที่ |0⟩ หรือ |1⟩? หมายความว่า f เป็น constant หรือ balanced?

กลุ่ม 24 — Deutsch's Oracle (f(x) = 0)

python

```
NUM_QUBITS = 2
layers = [
    [("x",  [1])],
    [("h",  [0]), ("h",  [1])],
    # Layer 3: Oracle f(x)=0 ⟶ ไม่ทำอะไร (identity)
    [("h",  [0])],
]
```

คำถาม: เทียบผลกับกลุ่ม 23 — q0 ต่างกันอย่างไร? สรุป Deutsch's Algorithm ได้อะไร?

กลุ่ม 25 — Phase Kickback

python

```
NUM_QUBITS = 2
layers = [
    [("h",  [0])],                # control ใน superposition
    [("x",  [1])],                # target = |1⟩ (eigenstate ของ Z)
    [("cz", [0, 1])],                # CZ: phase kickback เกิดที่ q0!
    [("h",  [0])],                # อ่านผล phase
]
```

คำถาม: Phase kickback คืออะไร? CZ กระทบ q0 อย่างไรทั้งที่ q0 เป็น control?

กลุ่ม 26 — Quantum Interference

python

```
NUM_QUBITS = 1
```

```
layers = [
    [("h",  [0])],                  # สร้าง superposition
    [("rz", [0], np.pi)],            # เพิ่ม phase π (= Z gate)
    [("h",  [0])],                  # interference!
    [("ry", [0], np.pi/4)],          # small rotation
]
```

คำถาม: ทำไม H⟶Z⟶H ทำให้ได้ |1⟩? อธิบาย constructive/destructive interference

กลุ่ม 27 — Amplitude Amplification (ย่อส่วน)

python

```
NUM_QUBITS = 2
layers = [
    [("h",  [0]), ("h",  [1])],        # equal superposition
    [("cz", [0, 1])],                  # mark |11⟩ ด้วย phase
    [("h",  [0]), ("h",  [1])],        # ⟵ ส่วนหนึ่งของ Grover diffusion
    [("z",  [0]), ("z",  [1])],        # ⟵ ส่วนที่ 2 ของ diffusion
]
```

คำถาม: Probability ของ |11⟩ เพิ่มขึ้นไหมหลัง amplification? ดูจาก statevector

กลุ่ม 28 — Quantum Fourier Transform (2 qubits)

python

```
NUM_QUBITS = 2
layers = [
    [("x",  [0]), ("x",  [1])],        # เริ่มจาก |11⟩
    [("h",  [1])],                     # H บน MSB
    [("rz", [1], np.pi/2)],            # Controlled-R ≈ CP (simplified)
    [("h",  [0])],                     # H บน LSB + SWAP
]
```

คำถาม: QFT เปลี่ยน computational basis ⟶ Fourier basis อย่างไร? ดูจาก probability

กลุ่ม 29 — Toffoli Gate Preparation

python

```
NUM_QUBITS = 3
layers = [
    [("x",  [0]), ("x",  [1])],        # ทั้ง control = |1⟩
    [("h",  [2])],                     # target ใน superposition
    [("cx", [0, 2])],                  # partial Toffoli
    [("cx", [1, 2])],                  # complete (simplified)
]
```

คำถาม: Toffoli gate ทำงานอย่างไร? ต้องการ control ทั้ง 2 ตัวเป็น |1⟩ จริงไหม?

กลุ่ม 30 — Fredkin (Controlled-SWAP)

python

```
NUM_QUBITS = 3
layers = [
    [("x",  [0])],                     # control = |1⟩
    [("ry", [1], np.pi/3), ("rx", [2], np.pi/4)],   # q1, q2 ต่างกัน
    [("cx", [1, 2]), ("cx", [2, 1])],    # SWAP part (simplified)
```

```
    [("cx", [1, 2])],                # complete SWAP
]
```

คำถาม: q1, q2 สลับกันหลัง Layer 4 ไหม? (เทียบ Bloch sphere กับ Layer 1)

กลุ่มที่ 31-40: Application Circuits (2-4 qubits)

โจทย์: ประยุกต์ quantum circuit กับปัญหาจริง เน้น: เชื่อมโยง Bloch sphere กับความหมายทาง application

กลุ่ม 31 — Quantum Random Number Generator

python

```
NUM_QUBITS = 3
layers = [
    [("h",  [0]), ("h",  [1]), ("h",  [2])],
    [("rx", [0], np.pi/7), ("ry", [1], np.pi/5), ("rz", [2], np.pi/3)],
    [("cx", [0, 1]), ("cx", [1, 2])],
    [("h",  [0]), ("h",  [1]), ("h",  [2])],
]
```

คำถาม: Probability distribution แบน (uniform) แค่ไหน? อะไรทำให้สุ่มมากขึ้น?

กลุ่ม 32 — Dense Coding (2 classical bits via 1 qubit)

python

```
NUM_QUBITS = 2
# ส่งข้อความ "10" ⟶ Alice ใช้ Z gate
layers = [
    [("h",  [0])],                   # สร้าง Bell pair
    [("cx", [0, 1])],                # entangle
    [("z",  [0])],                   # Alice encodes "10"
    [("cx", [0, 1]), ("h", [0])],        # Bob decodes
]
```

คำถาม: ถ้าเปลี่ยน Layer 3 เป็น X, Y, I (identity) จะส่งข้อความอะไรได้?

กลุ่ม 33 — Error Detection (bit flip)

python

```
NUM_QUBITS = 3
layers = [
    [("ry", [0], np.pi/3)],            # q0 = arbitrary state
    [("cx", [0, 1]), ("cx", [0, 2])],   # encode: copy ไป q1, q2
    [("x",  [1])],                     # ERROR: bit flip บน q1!
    [("cx", [0, 1]), ("cx", [0, 2])],   # syndrome extraction
]
```

คำถาม: ดู statevector หลัง Layer 4 — ระบุได้ไหมว่า error อยู่ที่ qubit ไหน?

กลุ่ม 34 — Error Detection (phase flip)

python

```
NUM_QUBITS = 3
layers = [
    [("h",  [0])],
    [("cx", [0, 1]), ("cx", [0, 2])],
    [("z",  [2])],                     # ERROR: phase flip บน q2!
```

```
    [("cx", [0, 1]), ("cx", [0, 2])],
]
```

คำถาม: Phase error ตรวจจับยากกว่า bit flip อย่างไร? Bloch sphere บอกอะไรได้บ้าง?

กลุ่ม 35 — Quantum Coin Flip Game

python

```
NUM_QUBITS = 2
# q0 = coin, q1 = player's strategy
layers = [
    [("h",  [0])],                  # flip coin
    [("cx", [0, 1])],               # player entangles
    [("h",  [0])],                  # flip again
    [("cx", [0, 1]), ("h", [1])],   # player's final move
]
```

คำถาม: Player สามารถ "โกง" ได้ด้วย quantum strategy ไหม? P(win) เท่าไหร่?

กลุ่ม 36 — Quantum Adder (1+1 bit)

python

```
NUM_QUBITS = 3
# q0 = input A, q1 = input B, q2 = carry
layers = [
    [("x",  [0]), ("x",  [1])],     # A=1, B=1 ⟶ should get sum=0, carry=1
    [("cx", [0, 2])],               # carry step 1
    [("cx", [1, 2])],               # carry step 2
    [("cx", [0, 1])],               # sum
]
```

คำถาม: 1+1 = 10 ในเลขฐานสอง — q1 (sum) และ q2 (carry) ถูกต้องไหม?

กลุ่ม 37 — Entanglement Swapping

python

```
NUM_QUBITS = 3
layers = [
    [("h",  [0])],
    [("cx", [0, 1])],               # Bell pair: q0-q1
    [("cx", [1, 2]), ("h", [1])],   # BSM on q1 ⟶ swap entanglement
    [("cx", [1, 2]), ("cz", [0, 2])],   # corrections
]
```

คำถาม: หลัง Layer 4 — q0 กับ q2 entangle กันไหม ทั้งที่ไม่เคย interact โดยตรง?

กลุ่ม 38 — Quantum Classifier (simple)

python

```
NUM_QUBITS = 2
# q0 = data qubit (encode angle), q1 = label qubit
layers = [
    [("ry", [0], np.pi/4)],         # encode data point θ1
    [("cx", [0, 1])],               # entangle with label
    [("ry", [1], np.pi/3)],         # trainable parameter
    [("rz", [0], np.pi/6)],         # trainable parameter
```

]

คำถาม: ถ้าเปลี่ยนมุม Layer 1 (data) ⟶ probability ของ q1 เปลี่ยนอย่างไร? นี่คือ classification

กลุ่ม 39 — Bernstein-Vazirani (secret = 11)

python

```
NUM_QUBITS = 3
# secret string s = "11", f(x) = s·x mod 2
layers = [
    [("x",  [2])],                    # ancilla = |1⟩
    [("h",  [0]), ("h",  [1]), ("h",  [2])],
    [("cx", [0, 2]), ("cx", [1, 2])],   # Oracle for s=11
    [("h",  [0]), ("h",  [1])],         # Hadamard ⟶ read secret
]
```

คำถาม: หลัง Layer 4 — q0 และ q1 อยู่ที่ |1⟩ ทั้งคู่ไหม? ถ้า secret = "01" ต้องแก้ oracle อย่างไร?

กลุ่ม 40 — Simon's Problem (ย่อส่วน, period s=11)

python

```
NUM_QUBITS = 4  # 2 input + 2 output
layers = [
    [("h",  [0]), ("h",  [1])],                    # superposition inputs
    [("cx", [0, 2]), ("cx", [1, 3])],                # copy input ⟶ output
    [("cx", [0, 3]), ("cx", [1, 2])],                # XOR with period s=11
    [("h",  [0]), ("h",  [1])],                     # Hadamard inputs
]
```

คำถาม: ดู probability ของ q0, q1 หลัง Layer 4 — สามารถหา period s ได้อย่างไร?